

[Short Paper] XQdrant: In-Database Attribution and Masked-Distance Subspace Search over HNSW

Moetez Fradi
moetez.fradi@insat.ucar.tn
INSAT
Tunis, Tunisia

Abstract

Vector databases return opaque scalar similarities over high-dimensional embeddings, giving practitioners no coordinate-level explanation of a match and no way to restrict HNSW traversal itself to a dynamic focus subspace. We present **XQdrant**, an extension of the Qdrant engine addressing both problems, and we report results for each honestly rather than only where they are favorable.

Attribution is unambiguously positive: `with_dims_explained` computes exact top- m per-dimension score decomposition in-database, avoiding client-side vector egress, and is recall-neutral against vanilla Qdrant while running $\sim 1.8\times$ faster than post-query extraction at encoder-scale widths ($D \in \{768, 1536\}$).

Masked-distance HNSW traversal is a mixed result, and we treat that as the paper’s second contribution: a systems-level characterization of *why* subspace-restricted traversal is harder than it looks. Naive all-layer masking collapses recall (SIFT1M mean Recall@10 ≈ 0.012 at $D_{\text{sub}}/D = 0.1$) because HNSW’s graph edges encode full-space proximity. A hybrid design that keeps full-distance routing on coarse layers and masks only near the base layer (`mask_from_layer`) restores navigability (SIFT mean recall 0.90–0.995 for cutoffs ≥ 1) but does *not* yield end-to-end latency wins, because graph descent remains dominated by I/O and candidate-set bookkeeping rather than distance arithmetic; a dedicated SIMD gather kernel wins microbenchmarks (1.1–1.8 $\times$ over repack-then-score) without changing this. We further show that subspace/full-space k -NN *coherence* predicts when a cheap verification pass can recover full-space recall, and that an α -weighted blend of full and masked scores is strictly dominated by a hard layer cutoff.

We release a reproducible harness and an experiment index so each of these positive and negative findings can be independently reproduced or cited.

Keywords

vector databases, HNSW, approximate nearest neighbor, explainability, feature attribution, subspace search, masked distance, SIMD

1 Introduction

High-dimensional dense embeddings underpin retrieval-augmented generation (RAG), recommendation, and semantic search [4, 8, 10]. Production engines traverse HNSW graphs [6], collapsing every comparison into a single scalar. This creates two gaps that motivate this paper.

Explainability. Practitioners cannot see which coordinates drove a hit without fetching full vectors and recomputing per-dimension terms client-side, an expensive, egress-bound pattern.

Subspace routing. Many workloads care about a dynamic focus set $\mathcal{F}$ of coordinates (concept probes, ablations, modality masks), yet stock HNSW always evaluates full D -dimensional distances during graph descent. Rescoring on $\mathcal{F}$ after full-vector preselection improves semantic specificity but, as we show, cannot reduce traversal cost.

XQdrant addresses the first gap successfully and investigates the second rigorously. For attribution, `with_dims_explained` attaches exact top- m coordinate terms to search results in-database. For subspace routing, `nearest.focus` with `masked=true` evaluates reduced distances *during* HNSW traversal, not just after it, via a hybrid layer cutoff, gather-based SIMD kernels, an optional verify/overfetch pass, and a full/masked score blend. Where the second contribution falls short of a latency win, we do not paper over it: we identify the mechanism (navigability vs. arithmetic cost) and report exactly which configurations are and are not worth shipping.

Contributions

- **In-database attribution (positive result).** Exact score decomposition with order-statistic top- m selection; recall-neutral vs. vanilla Qdrant and $\sim 1.8\times$ faster than post-query extraction.
- **Masked-distance HNSW traversal (mechanism-level negative result).** A hybrid layer-cutoff design that restores navigability where naive masking collapses it, but does not translate into end-to-end speedup; and an explanation, grounded in coherence and kernel microbenchmarks, of why.
- **Coherence as a predictive diagnostic.** Subspace/full-space k -NN overlap predicts when a cheap verify pass can recover full-space recall and when it cannot, independent of the traversal strategy used.
- **Reproducible artifacts.** A public experiment index and harness (Section 8) mapping every claim above to a canonical script and dataset.

2 Background

2.1 HNSW and Contiguous SIMD

HNSW maintains multi-layer proximity graphs [6]. Queries greedily descend with candidate heap size `ef_search`, then extract top- k at the base layer. Distance kernels assume contiguous f32 slices (AVX2/NEON), so a non-contiguous focus set must either (i) repack selected coordinates into a scratch buffer before scoring, or (ii) gather them directly via SIMD gather loads.

2.2 Score Decomposition

For Dot product, $s(\mathbf{q}, \mathbf{v}) = \sum_i q_i v_i$; Cosine applies the same decomposition after ℓ_2 normalization; Euclidean and Manhattan

admit analogous additive terms. Attribution is therefore exact accounting of the engine’s own metric, not a surrogate explanation model such as SHAP [5] or LIME [9].

2.3 Subspace Coherence

Let $N_k^{\text{full}}(q)$ and $N_k^{\mathcal{F}}(q)$ denote exact k -NN sets under the full metric and under focus set $\mathcal{F}$. We define *coherence* as $|N_k^{\text{full}} \cap N_k^{\mathcal{F}}|/k$, and refer to the diagnostic built around this quantity as **C1** in Section 5.6. Coherence is the quantity that determines, independent of any particular traversal strategy, whether full-space neighbors can even in principle be recovered from subspace-guided search; it recurs throughout Section 5 as the explanatory variable behind several otherwise-surprising results.

3 In-Database Attribution

XQdrant injects attribution at the collection coordinator after shard merge, with an early return when `with_dims_explained` is absent, and zero overhead on the unmodified hot path. Per explained hit, Algorithm 1 materializes D terms and selects the top- m by absolute contribution in average $O(D)$ time via Rust’s order-statistic selection (`select_nth_unstable`).

Algorithm 1 (In-database top- m attribution).

Input: query q , vector v , D , m , metric δ .

1. Normalize operands if required by δ .
2. For $i = 0, \dots, D - 1$: append $(i, \text{TERM}_i(q, v, \delta))$.
3. Return order-statistic top- m by $|t|$, sorted descending.

Figure 1: In-database top- m feature attribution.

`nearest.focus` without masking still performs full HNSW preselection and rescores on $\mathcal{F}$ afterward, useful when ranking must respect a focus subspace, but, as Section 5.2 shows, not a traversal-cost reduction. That gap is exactly what motivates masked traversal (Section 4).

4 Masked-Distance HNSW: Design Space

We consider three traversal-level strategies for evaluating distance only over a focus subspace $\mathcal{F}$ during HNSW descent, plus two supporting mechanisms (a kernel choice and a recall-recovery pass) that apply across all three. Table 1 fixes the naming used for the rest of the paper.

Table 1: Naming used throughout Sections 4–5.

Name	What it is
M1	Naive: mask every HNSW layer
M2	Hybrid: full distance for layers $\geq L$, masked below (<code>mask_from_layer = L</code>)
M3	M2 plus an α -weighted blend of full/masked scores on masked layers
K1	Kernel choice: repack-then-score vs. SIMD gather, orthogonal to M1–M3
V1	Verify/overfetch: optional post-traversal full-distance rescore, orthogonal to M1–M3
C1	Coherence diagnostic (Section 2.3), used to interpret M1–M3 and V1

4.1 API Surface

Focus parameters ride inside the existing query body:

```
"focus": {
  "dims": [...], "masked": true,
  "mask_from_layer": 1,    // M2/M3 hybrid cutoff
  "verify": true,          // optional full rescore
  "alpha": 0.25            // M3 blend weight
}
```

Constraints mirror existing dense local-search paths: no quantization/turbo, no simultaneous `with_dims_explained` on masked paths, and fail-fast if `mask_from_layer`, `verify`, or `alpha` are set without `masked=true`.

4.2 M1: Naive All-Layer Masking

With `masked=true` and no cutoff, every HNSW comparison during descent uses only coordinates in $\mathcal{F}$. This is the cheapest semantic change to implement, and the one most likely to be tried first, which is precisely why we report it as a cautionary baseline rather than a proposal: it destroys navigability whenever the focus metric disagrees with the full-space geometry the graph was built on (Section 5.3).

4.3 M2: Hybrid Layer Cutoff

Layer ℓ uses the masked scorer iff $\ell < L$ for cutoff $L = \text{mask_from_layer}$; otherwise full distance is used. $L=0$ recovers plain (unmasked) nearest search; a large L recovers M1. The implementation builds dual raw scorers inside `FilteredScorer` and switches via `set_layer` at layer boundaries; visited-set and heap logic are unchanged, so this composes with the rest of the query pipeline for free.

4.4 M3: Weighted Blend

On masked layers, $\text{score} = \alpha \cdot s_{\text{full}} + (1 - \alpha) \cdot s_{\text{focus}}$; coarse (unmasked) layers are unaffected. Note that blending computes *both* distances, so it cannot reduce arithmetic cost on its own; it exists purely as a routing-bias ablation to test whether it recovers recall at a fixed cutoff more cheaply than raising L .

4.5 K1: Kernel Choice: Repack vs. Gather

Masked scoring can copy focus dims into a contiguous scratch buffer before calling the existing SIMD kernels (*repack*), or gather them directly in-register (AVX2 gather for Dot/Cosine/Euclidean on x86; scalar fallback elsewhere). We add the gather path, selectable via environment variable `XQDRANT_MASKED_KERNEL`, and treat kernel choice as orthogonal to M1–M3: any of the three can use either kernel.

4.6 V1: Verify and Overfetch

`verify=true` re-scores returned candidates with full distance after traversal, reusing the existing quantization-rescore hook. At overfetch $m=1$ (i.e., *limit* = k) this only reorders a size- k set, so $\text{Recall}@k$ is provably unchanged; it is a correctness/ordering feature, not a recall-recovery one. Overfetch with $m > 1$ (*limit* = $m \cdot k$, keep top- k after full rescore) turns masked traversal into a candidate *filter* and can recover recall, at a latency cost we quantify in Section 5.7.

5 Experimental Evaluation

We structure this section as an argument rather than a list of runs: each subsection’s result motivates the next question. Table 2 previews where this argument ends, so it can be read as a map before the detailed results.

5.1 Setup

Hardware and protocol. Both suites run on the *same* Linux x86_64 host via HTTP against local release binaries (vanilla Qdrant and XQdrant), with gather as the default masked kernel unless a kernel comparison is the subject. Absolute latencies are therefore comparable across suites. Unless a caption says otherwise, each reported configuration is the mean of $T=5$ independent trials that resample the timed query set (and, where applicable, the focus-dimension subset) under distinct seeds; we quote sample standard deviation inline or as error bars. Within each trial, latency summaries use the per-query p50 as before. Orchestration lives in the unified paper bench script (Section 8). Kernel microbenchmarks (Section 5.5) are offline ns/op measurements on the same host class; the M3 α -sweep figure is a canonical SIFT cutoff $\times \alpha$ panel whose qualitative ranking ($\alpha=0$ best) matches the hybrid M2 result.

Attribution suite. Synthetic corpora with $N=10k$ vectors and $Q=500$ queries at $D \in \{768, 1536\}$; baselines are vanilla Qdrant, post-query client-side top- m extraction, and XQdrant.

Masked-distance suite. Corpora: SIFT1M ($D=128$) and synthetic data with $N=50k$ at $D \in \{768, 1536\}$, $Q=500$, $k=10$, and $ef=128$. Scripts, CSVs, and the experiment index for every claim below are linked from Section 8.

5.2 Attribution and Focus Rescoring Are Not the Same Problem

Figure 2 shows `with_dims_explained` preserves the latency-recall frontier vs. vanilla Qdrant at $D=768$ (mean Recall@10 gap < 0.01 across $T=5$ trials). Figure 3 shows in-database attribution holds a flat mean p50 of 3.85–4.02 ms (± 0.07) across $m \in \{1, \dots, 50\}$ while post-query extraction sits at 6.87–7.03 ms ($\approx 1.78\times$). This confirms attribution as a clean systems win. But Figure 4 shows that nearest *focus rescoring*, the natural first attempt at subspace search, using the same full-preselection machinery, yields speedup < 1 (0.80–0.84). Rescoring reduces the dimensionality of the arithmetic but not the traversal it depends on. This negative result is what motivates attacking the traversal itself, which is the subject of the rest of this section.

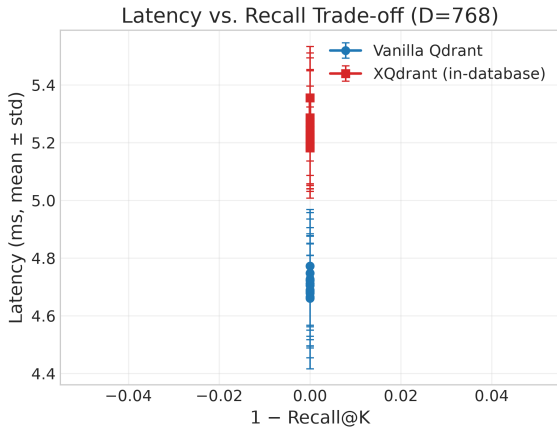


Figure 2: Attribution suite: latency-recall at $D=768$ with `with_dims_explained` ($m=10$). Points are mean $\pm$ std over $T=5$ query-sample trials.

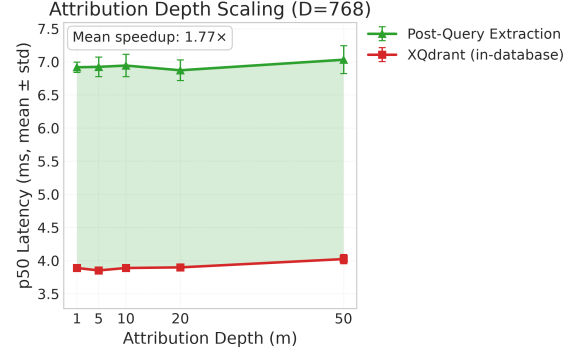


Figure 3: Attribution depth: in-DB vs. post-query extraction ($D=768$). Mean $\pm$ std over $T=5$ trials.

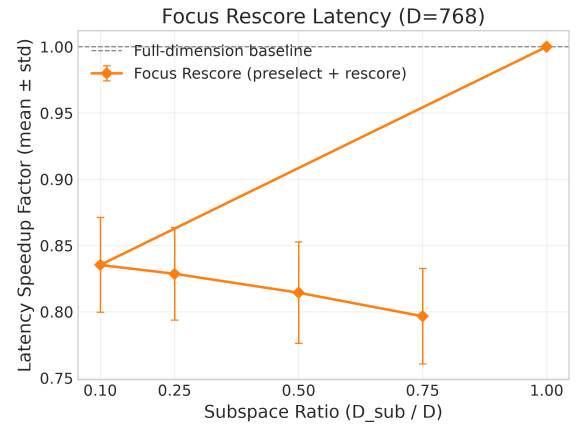


Figure 4: Focus rescoring speedup $T_{\text{full}}/T_{\text{sub}}$ ($D=768$): values < 1 show rescoring alone does not accelerate search (mean $\pm$ std, $T=5$).

5.3 M1: Naive Masking Breaks Navigability

If rescoring cannot help, the natural next attempt is to mask distance *during* traversal (M1). Figure 5 shows this fails sharply on SIFT1M: mean Recall@10 against full-space ground truth is 0.012, 0.077, 0.250, 0.490, and 0.998 (± 0.002 –0.014) at subspace ratios 0.1, 0.25, 0.5, 0.75, and 1.0. This is the navigability breakdown predicted in Section 2: graph edges encode full-space proximity, and masked greedy descent has no guarantee of reaching the region containing the true neighbors once that proximity assumption is violated.

5.4 M2: A Hybrid Cutoff Restores Navigability, Not Latency

M2 keeps full-distance routing on coarse layers, hypothesizing that most of navigability is determined there. Figure 6 confirms this: on SIFT, cutoffs $L \geq 1$ recover 0.90–0.995 mean recall (subspace-restricted ground truth; across-trial std ≤ 0.004), while $L=0$ tracks the low coherence floor. On high- D synthetic data the same cutoff lifts otherwise-collapsed cells (e.g., $D=768$, ratio 0.25: $\sim 0.01 \rightarrow \sim 0.20$) but plateaus near 0.15–0.22; a ceiling we explain via coherence in Section 5.6.

Critically, **end-to-end speedup vs. full search remains below 1 in every measured cell** (Figure 8), even using the gather kernel. Section 5.5 isolates why: the arithmetic savings

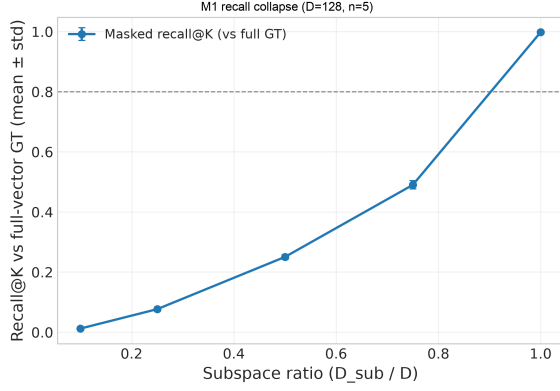


Figure 5: M1 on SIFT1M: Recall@10 vs. full-space ground truth as D_{sub}/D shrinks (mean $\pm$ std over $T=5$ trials).

from masking are real but small relative to the I/O and candidate-management cost that dominates HNSW wall-clock time.

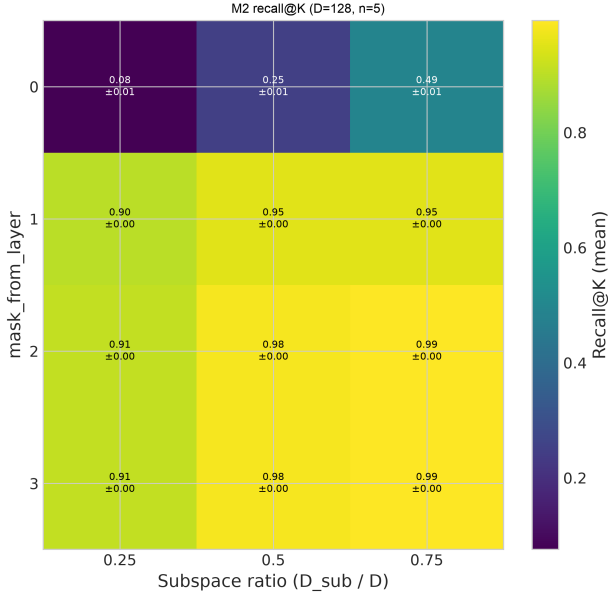


Figure 6: M2 on SIFT1M: recall heatmap over $\text{mask_from_layer} \times \text{subspace ratio}$, gather kernel (cell text: mean $\pm$ std).

5.5 K1: Kernel Choice Explains the Ceiling, Not the Floor

Figure 9 isolates the SIMD kernel question directly: gather is $\sim 1.24\times / 1.14\times / 1.42\times / 1.77\times$ faster than repack-then-score at $D_{\text{sub}} \in \{32, 128, 384, 768\}$ in ns/op microbenchmarks. Re-running the M2 sweep with gather instead of repack leaves recall unchanged and still produces **zero** end-to-end cells above $1\times$. This isolates the bottleneck to graph traversal overhead (candidate heap management, visited-set bookkeeping, memory access to non-contiguous graph pages) rather than the per-comparison arithmetic; the kernel is not the limiting factor, so faster kernels alone cannot close the gap identified in Section 5.4.

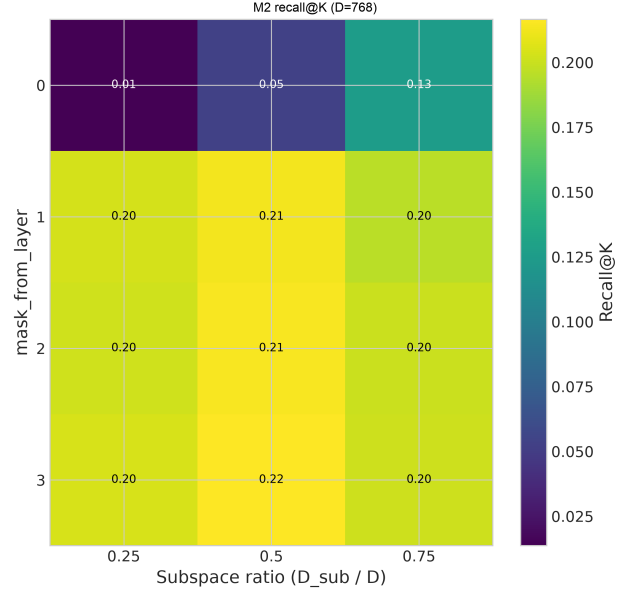


Figure 7: M2 on synthetic $D=768$: same sweep, showing the recall plateau discussed in Section 5.6.

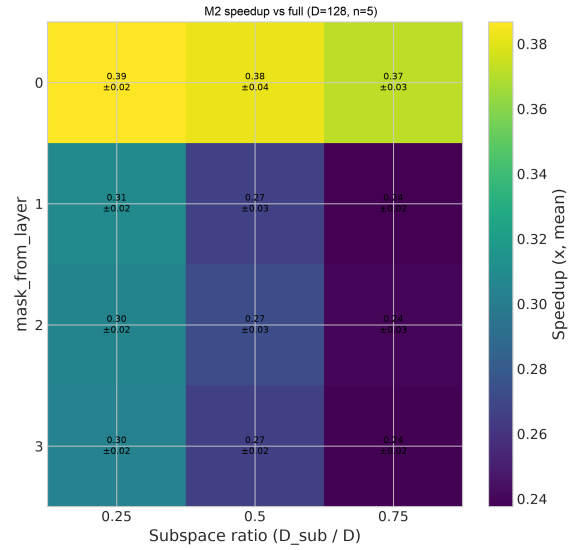


Figure 8: M2 SIFT end-to-end speedup vs. full search: no cell exceeds $1\times$, despite recovered recall (mean $\pm$ std).

5.6 C1: Coherence Explains the Recall Ceiling

Why does M2's recall plateau at 0.15–0.22 on high- D synthetic data (Section 5.4) rather than approaching 1? Figure 10 measures subspace/full-space exact top-10 overlap directly: it grows with ratio but stays low on synthetic $D=768$ ($\sim 0.007 \rightarrow 0.30$ from ratio 0.1 $\rightarrow$ 0.75); SIFT is higher (mean 0.013 $\rightarrow$ 0.490 from ratio 0.1 $\rightarrow$ 0.75 under $T=5$). This is a property of the *data and focus set*, not of the traversal algorithm: when the subspace and full-space neighbor sets barely overlap, no amount of navigability recovery can produce full-space-correct results, because the answers the two metrics agree on are simply different. This reframes Section 5.4's

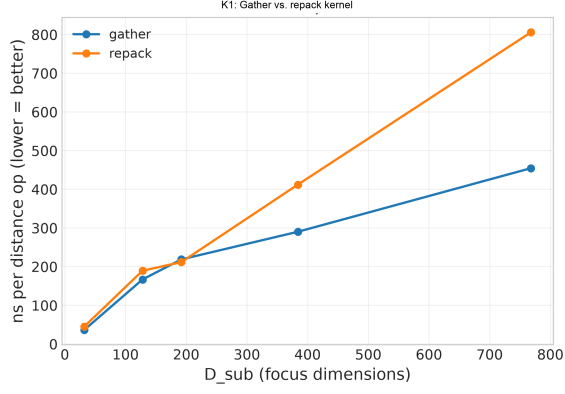


Figure 9: K1: Masked Dot-product kernel throughput, repack vs. gather.

plateau from an implementation limitation into an expected consequence of low coherence, and predicts when the verification pass in Section 5.7 can and cannot help.

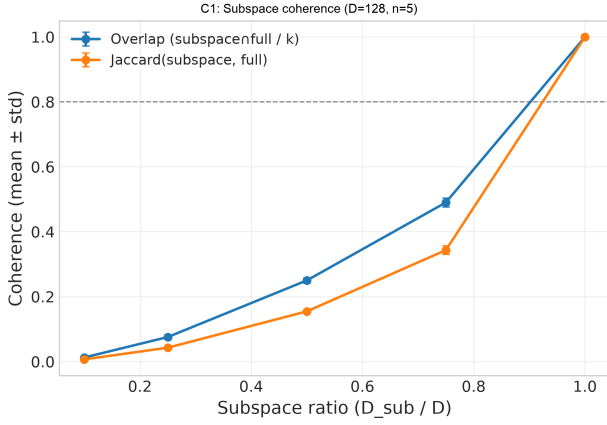


Figure 10: C1: Subspace↔full exact top-10 overlap vs. ratio, SIFT1M (mean±std, $T=5$).

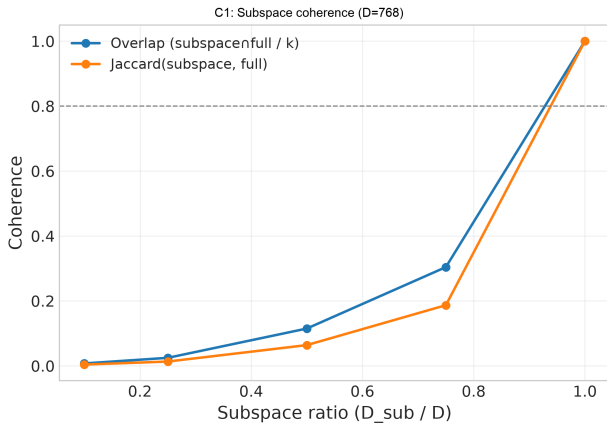


Figure 11: C1 on synthetic $D=768$: lower coherence throughput, consistent with the lower recall ceiling in Figure 7.

5.7 V1: Verify + Overfetch Trades Latency for Recall, Where Coherence Allows

Given that M2 cannot reach full-space recall on its own, can a post-traversal verification pass close the gap? At overfetch $m=1$, masked and verified Recall@10 are identical, as predicted in Section 4; reordering a fixed candidate set cannot change which candidates it contains. At $m=8$ on SIFT with hybrid $L=1$ (high coherence regime), full-space recall rises substantially: $0.26 \rightarrow 0.66$ at ratio 0.5 and $0.45 \rightarrow 0.89$ at ratio 0.75 (Figure 12), but at a cost of roughly +45 ms p50 (~ 1.5 ms $\rightarrow \sim 46$ ms); overfetching by $8\times$ and fully rescoring is not cheap. On high- D synthetic data (low coherence), recall stays $\leq \sim 0.19$ even at $m=8$: exactly the outcome Section 5.6 predicts, since no amount of overfetch recovers neighbors that the subspace metric never surfaces as candidates in the first place.

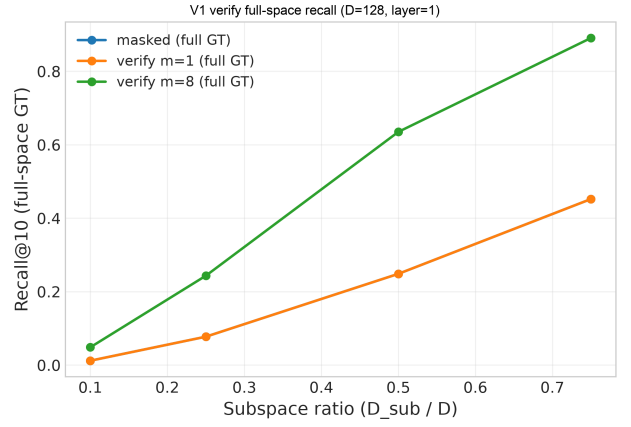


Figure 12: V1: Full-space Recall@10, masked vs. verify, at overfetch $m \in \{1, 8\}$, SIFT1M with hybrid $L=1$.

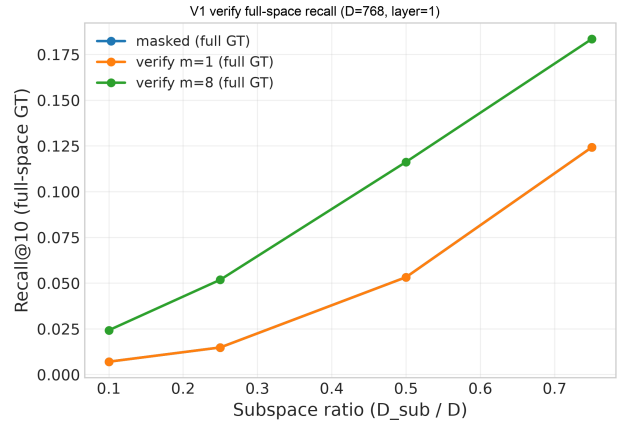


Figure 13: V1 on synthetic $D=768$: overfetch cannot rescue recall under low coherence.

5.8 M3: Blending Is Dominated by a Hard Cutoff

Finally, we test whether M3's continuous α can outperform M2's discrete layer cutoff at matched cost. Figure 14 shows that on SIFT at ratio 0.25, recall is maximized at $\alpha=0$ for every cutoff

tested (~ 0.84 for $L \geq 1$ on the canonical SIFT panel) and degrades monotonically toward the coherence floor as $\alpha \rightarrow 1$. Since $\alpha=0$ is exactly M2, blending adds API surface and computes both distance terms without ever improving on the simpler design. We do not recommend shipping M3.

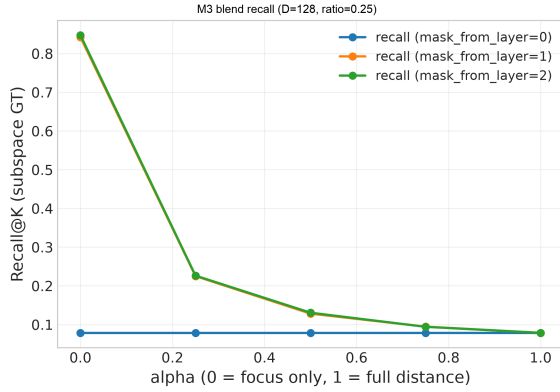


Figure 14: M3 on SIFT: recall vs. α for cutoffs $\{0, 1, 2\}$ (ratio 0.25, subspace ground truth; canonical panel).

5.9 Summary

Table 2 consolidates the keep/skip verdict along the argument chain of Sections 5.2–5.8.

Table 2: Keep / demote map for the masked-distance study.

Mechanism	Verdict	Evidence
Attribution (in-DB)	Keep	$\sim 1.8\times$ vs. post-query; recall-neutral
Focus rescoring only	Not accel.	Speedup < 1 (§5.2)
M1 naive mask	Baseline only	Full-GT collapse on SIFT (§5.3)
M2 hybrid cutoff	Keep	Restores navigability; e2e speedup still < 1
K1 gather kernel	Keep (micro)	1.1–1.8× ns/op; does not close e2e gap
C1 coherence	Keep	Predicts high- D / verify limits
V1, $m=1$	API only	Recall@ k provably unchanged
V1 + overfetch	Conditional	Helps on coherent data; $\sim +45$ ms; fails on low coherence
M3 blend	Skip	Best at $\alpha=0$, i.e. collapses to M2

6 Related Work

Vector databases. Milvus [10], Pinecone [7], and Qdrant [8] expose ANN with filters and quantization but not native per-coordinate decompositions or layer-aware masked metrics. Unlike those engines, we add in-database attribution and query-time masked HNSW scoring inside an existing production graph index.

ANN libraries. FAISS [3] and ScaNN [2] optimize large-scale search; explainability and dynamic subspace routing remain post-processing concerns external to the index. Unlike library-level ANN, we instrument traversal and scoring inside a serving engine and evaluate end-to-end latency, not only index throughput.

Explainable AI. SHAP and LIME [5, 9] approximate model behavior via sampling or local surrogates. Unlike those surrogate explainers, we exploit additive metric structure for deterministic, zero-variance attributions aligned with the engine’s own ranking, closer to in-database OLAP rollups [1] than to sampling-based XAI.

Dimension reduction and projected indexes. Static projections or multi-index layouts differ from *query-time* focus sets evaluated against an existing full-dimensional HNSW index. Unlike building a dedicated auxiliary index per focus set, which would sidestep the navigability problem of Section 5.3 but only for a registered palette of subspaces, we characterize what query-time masking of an unmodified full-space graph can and cannot deliver, and leave projected indexes to future work (Section 7).

7 Discussion and Conclusion

Shipping verdict. Keep in-database attribution and M2, keep K1 as an implementation detail, ship V1 only with overfetch on coherent data (Section 5.7), and do not ship M3 ($\alpha=0$ is M2).

Why no e2e speedup. Section 5.5 isolates the bottleneck to I/O and candidate management, not masked arithmetic. Query-time masking of an unmodified full-space HNSW therefore cannot yield wall-clock wins without a redesigned traversal structure. A dedicated projected index per registered focus set (Section 6) is the natural next step.

Limitations. No dedicated high- D M1 archive (collapse evidenced via M2’s low- L cells and C1); no visited-set divergence instrumentation; some high- D /verify panels predate the $T=5$ protocol and are treated as qualitative coherence support rather than primary latency claims.

XQdrant shows exact feature attribution can live in a Rust vector engine with a clear latency win, while masked-distance HNSW is a navigability tool rather than a latency win: M2 restores recall where M1 collapses, but subspace/full-space coherence caps what an unmodified full-space graph can recover. The public experiment index (Section 8) maps each positive and negative claim to a reproducible run.

8 Artifacts

Source code (XQdrant fork of Qdrant) and the experiment harness are public:

- Engine fork: <https://github.com/Moetez-Fradi/XQdrant>
- Experiment index, scripts, and canonical CSVs: https://github.com/Moetez-Fradi/XAI/tree/main/DB_systems

The harness repository is a submission stub at review time and will be populated with the full runners on acceptance (or earlier via CMT supplemental upload if requested). Planned contents: the unified orchestration script, per-mechanism runners, plotting entry points for M1–M3, K1, C1, and V1, and build/run notes for the reported XQdrant binary plus the vanilla Qdrant baseline. Neither repository uses access monitoring. A CMT zip upload is optional when these links are provided.

References

- [1] Jim Gray, Surajit Chaudhuri, et al. 1997. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Mining and Knowledge Discovery* 1, 1 (1997), 29–53.
- [2] Ruiqi Guo, Philip Sun, Erik Lindgren, et al. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*.
- [3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33.
- [5] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.

- [6] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [7] Pinecone Systems. 2023. Pinecone: Managed Vector Database for Production AI. <https://www.pinecone.io/>.
- [8] Qdrant Team. 2024. Qdrant: Vector Search Engine Documentation. <https://qdrant.tech/documentation/>.
- [9] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. doi:10.1145/2939672.2939778
- [10] Jianguo Wang, Xiaomeng Yi, Rentong Guo, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627. doi:10.1145/3448016.3457550

694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756